

图的遍历方式

深度优先搜索

深度优化遍历(**Depth First Search**), 也有称为 **深度优化搜索** , 简称为 **DFS** 。事实上, 我们在树的遍历中早已涉及DFS, 层序遍历、中序遍历和后序遍历都属于深度优先遍历的方式, 因为这些遍历方式本质上都归结于**栈**。为了讲清楚DFS, 我们先来看两个概念。

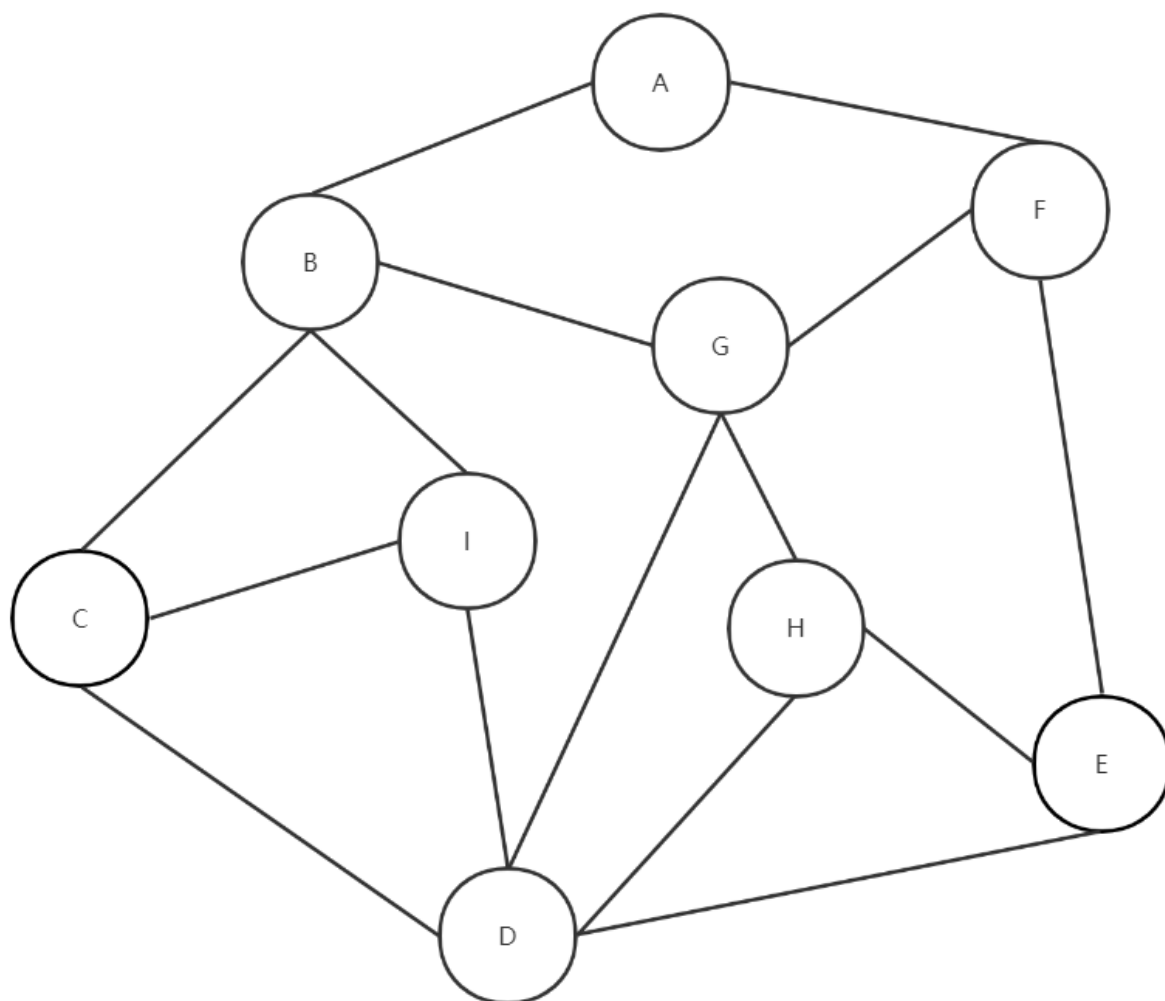
右手原则

在没有碰到重复顶点的情况下, 分叉路口始终是向右手边走, 每路过一个顶点就做一个记号。

左手原则

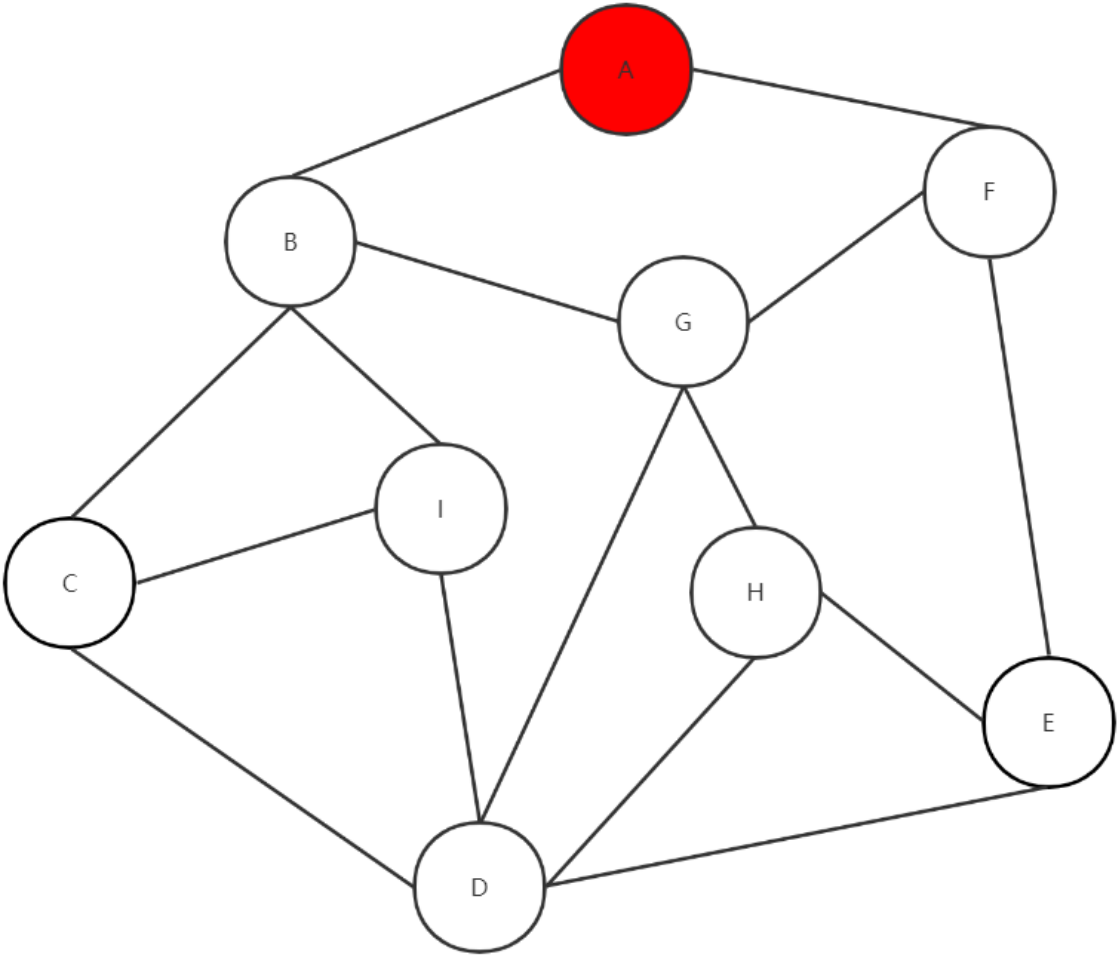
在没有碰到重复顶点的情况下, 分叉路口始终是向左手边走, 每路过一个顶点就做一个记号。

我们这边都以右手原则来进行深度优先搜索。话不多说, 直接开搜。

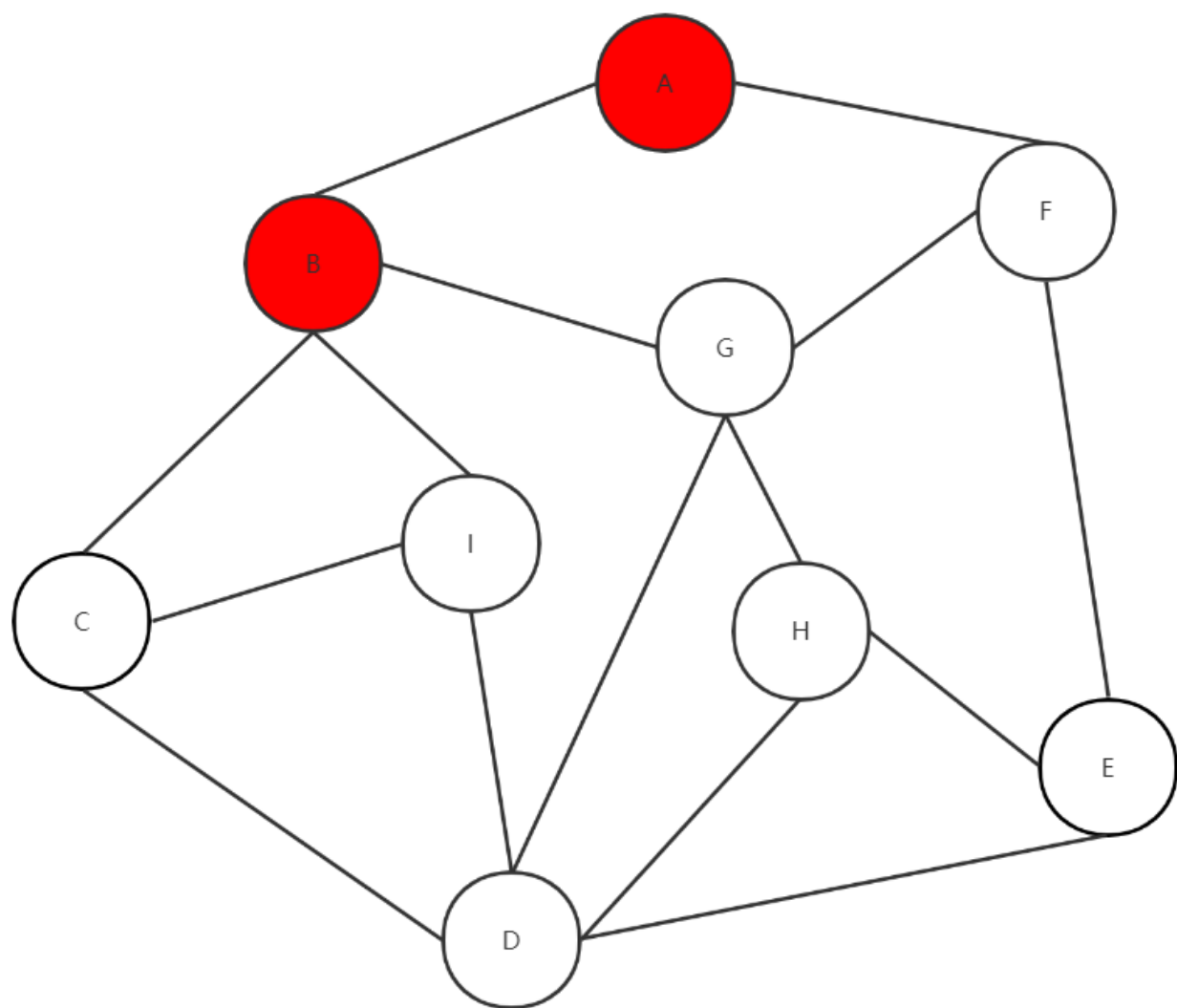


原则上, 我们可以从图中的任意一个顶点开始, 进行深度优先搜索, 假设我们从A顶点开始, 遍历过程如下。

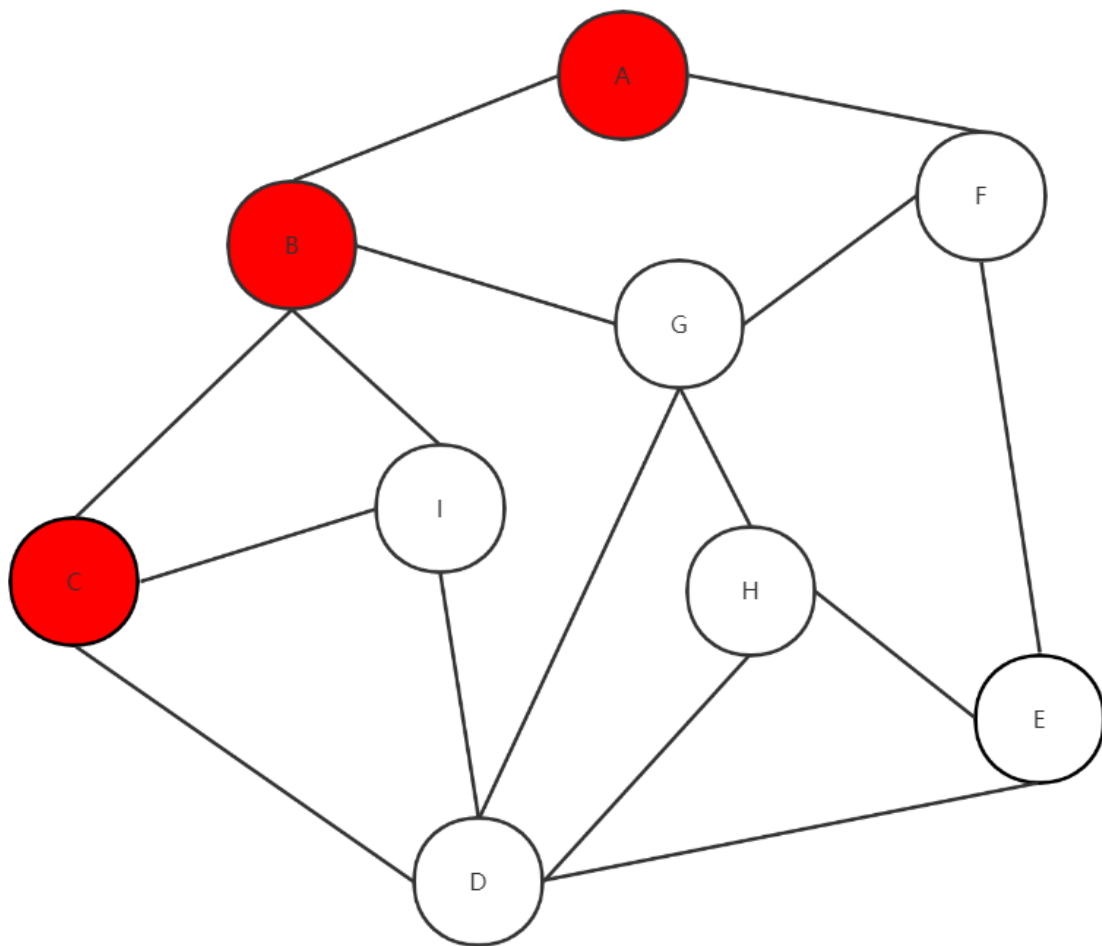
第一步：从顶点A开始，将顶点A标记为已访问结点



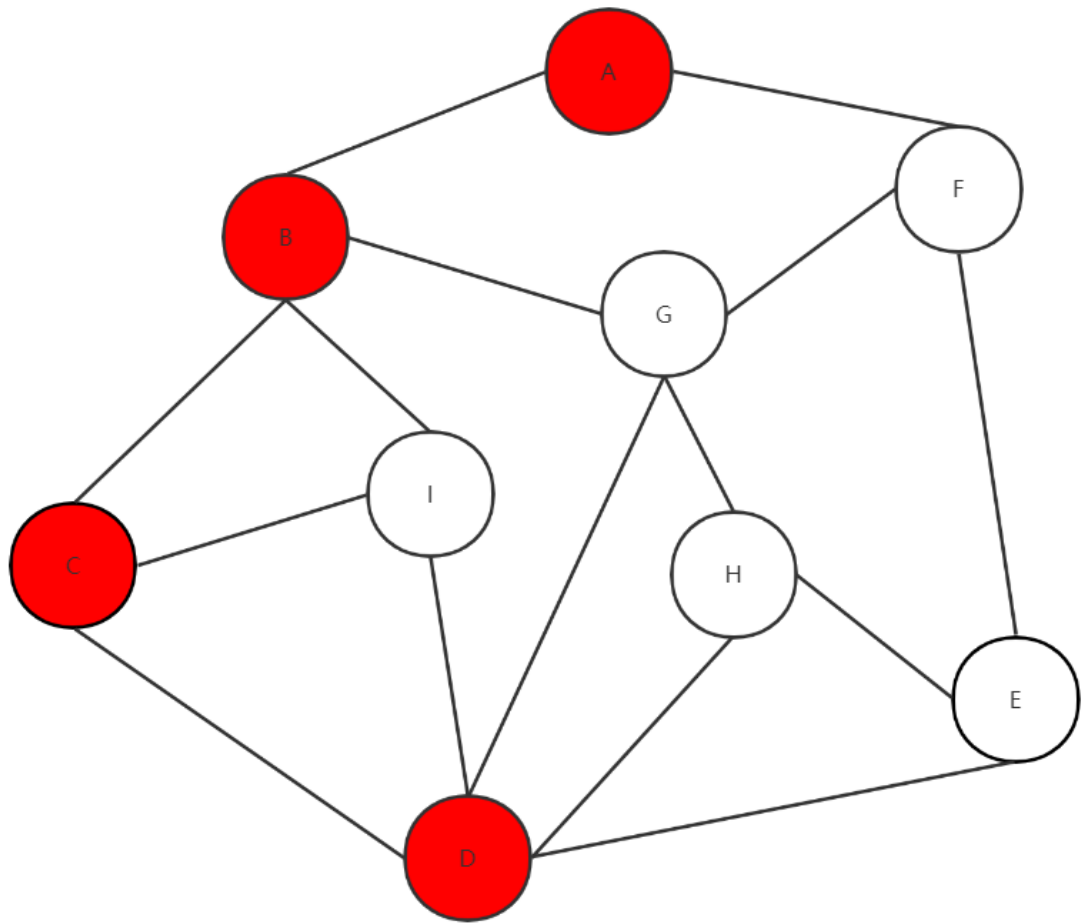
第二步：根据约定的右手原则，访问顶点B，并将B标记为已访问顶点。



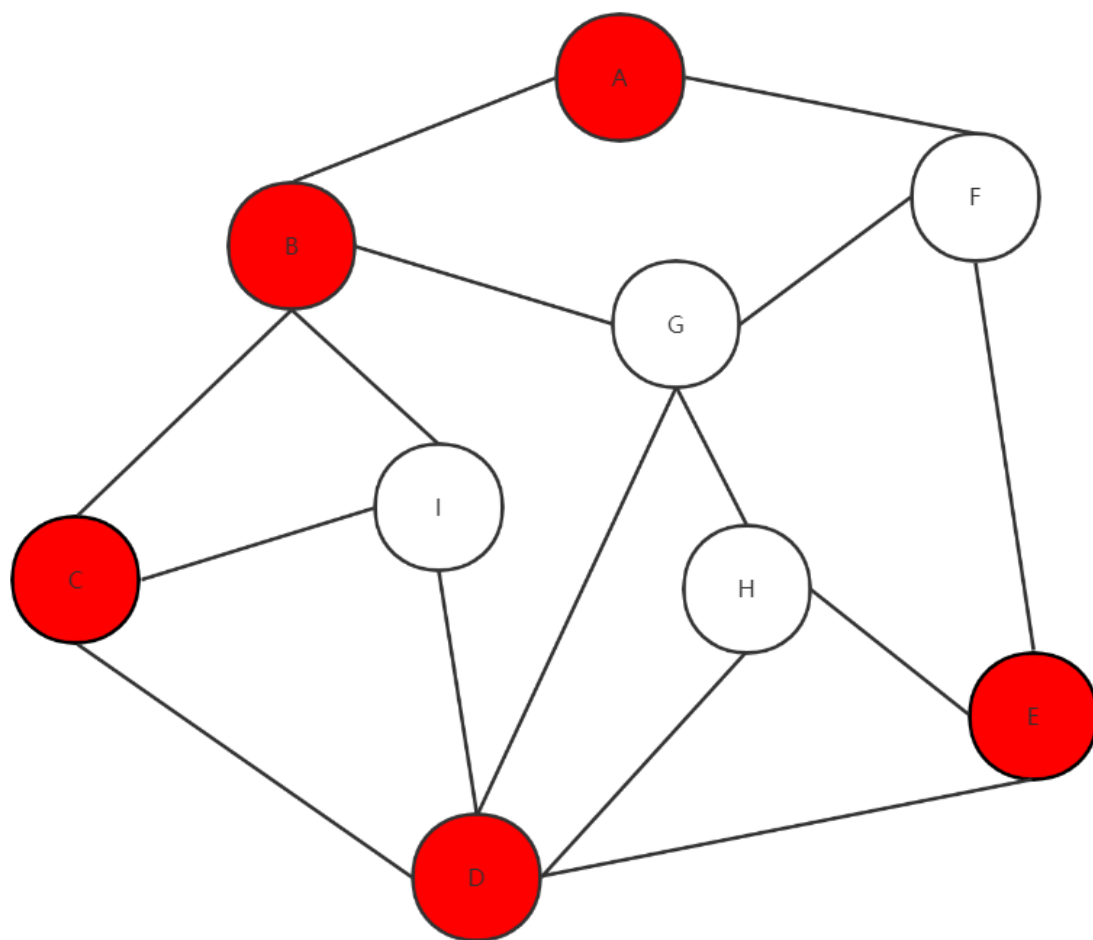
第三步：右手原则，访问C



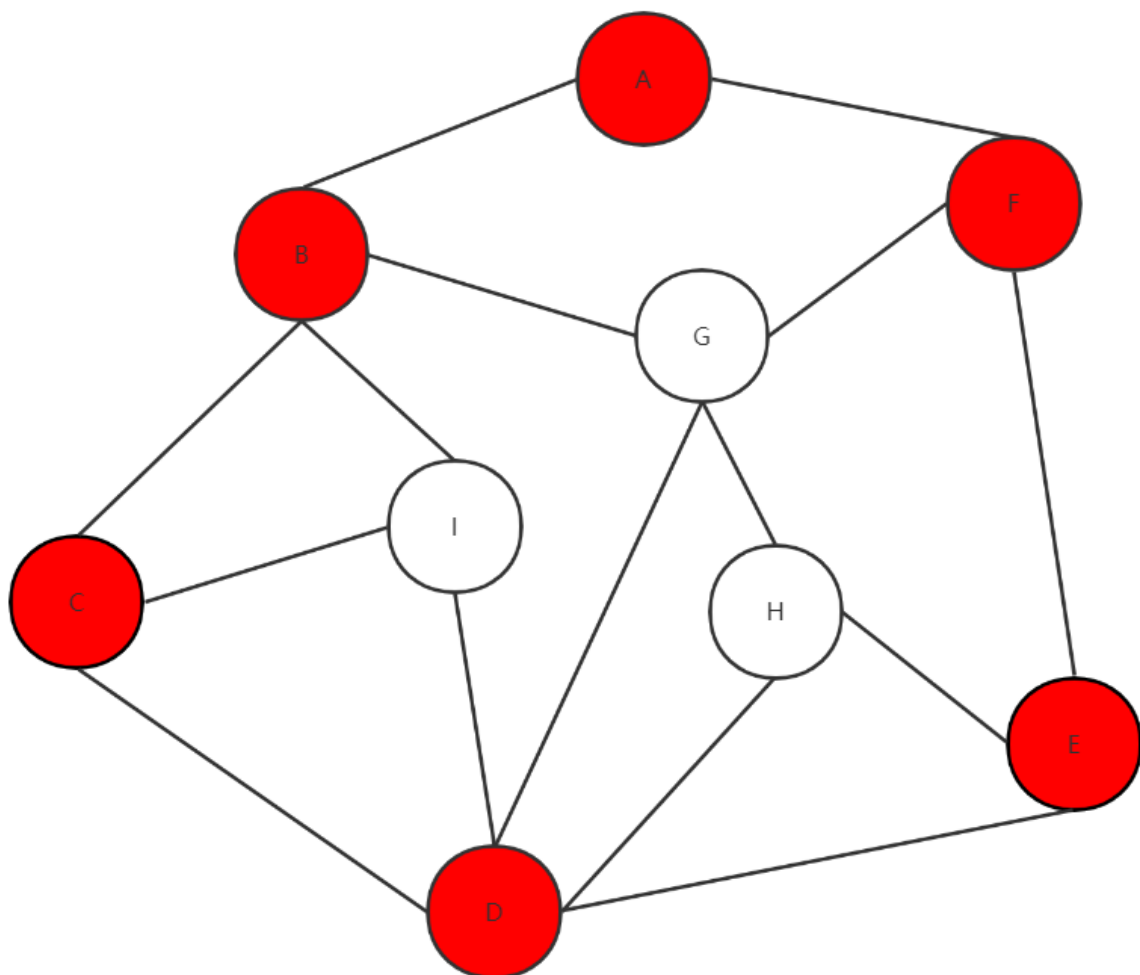
第四步：右手原则，访问顶点D



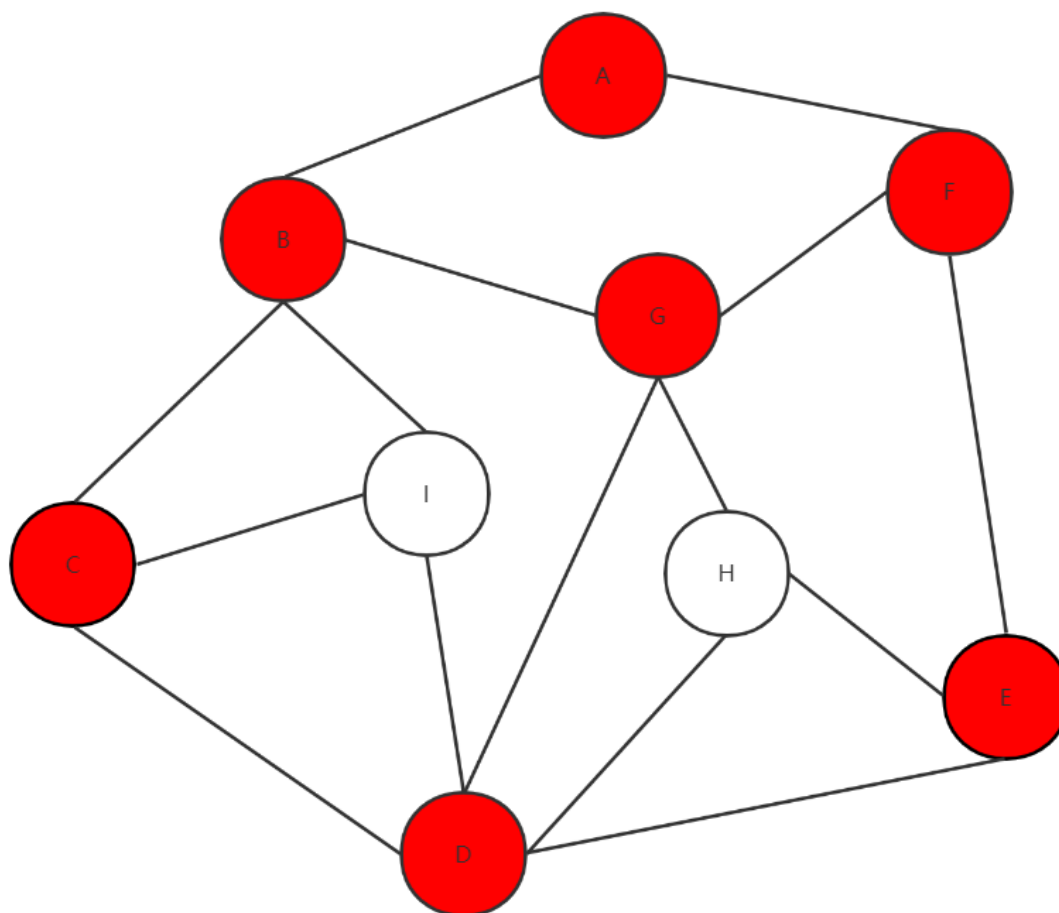
第五步：右手原则，访问E



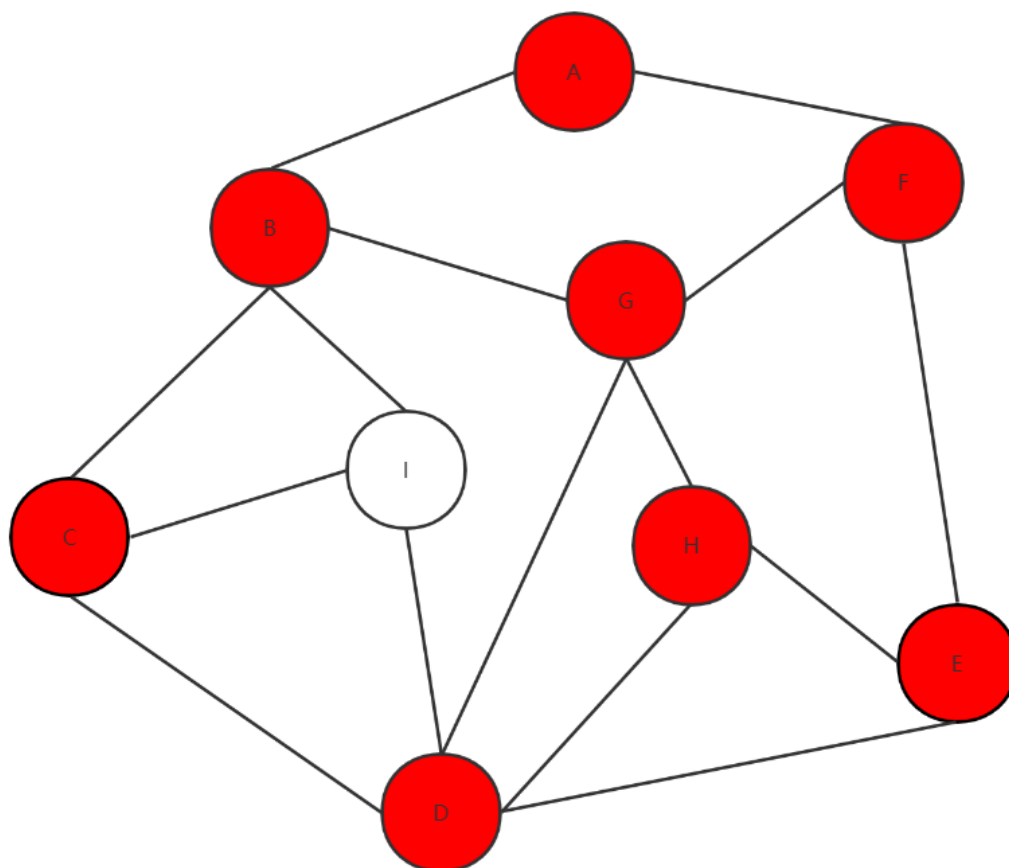
第六步：右手原则，访问F



第七步：右手原则，应该先访问顶点F的邻接顶点A，但发现A已被访问，则访问除A之外的最右侧顶点G。



第八步：右手原则，先访问顶点B，顶点B已被访问；再访问顶点D，顶点D已经被访问；最后访问顶点H。



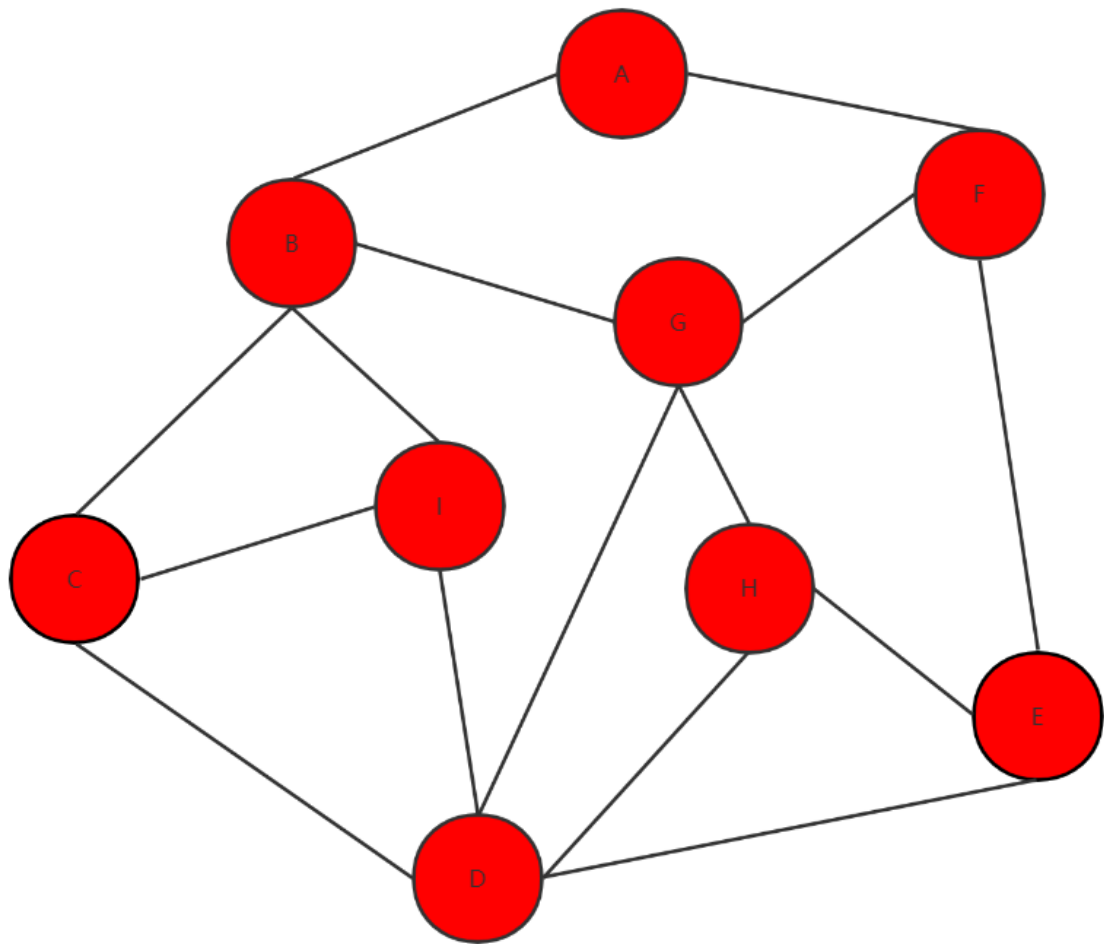
第九步：发现顶点H的邻接顶点均已被访问，则退回到顶点G；

第十步：顶点G的邻接顶点均已被访问，则退回到顶点F；

第十一步：顶点F的邻接顶点已被访问，则退回到顶点E；

第十二步：顶点E的邻接顶点均已被访问，则退回到顶点D；

第十三步：顶点D的邻接顶点I尚未被访问，则访问顶点I；

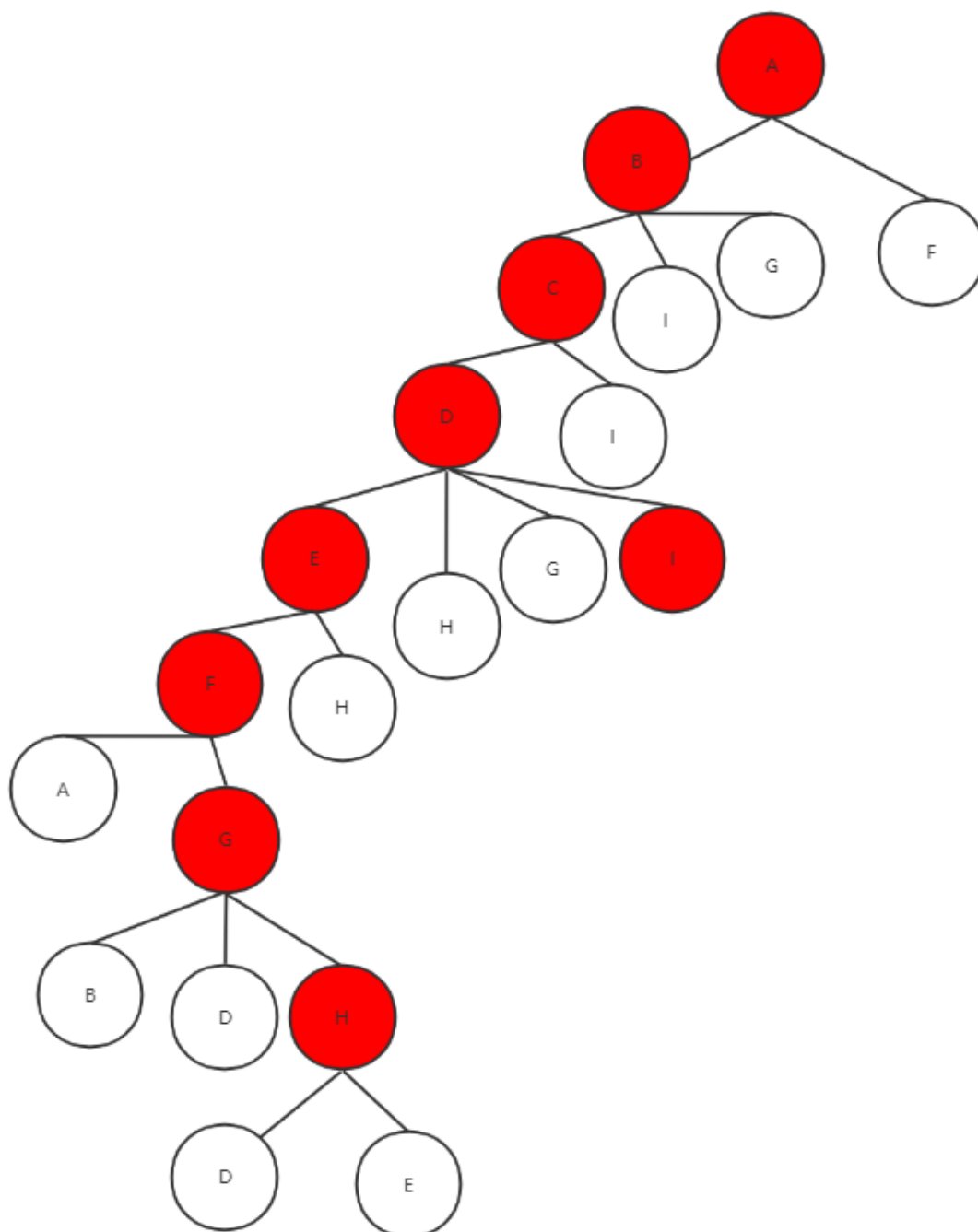


第十四步：顶点I的邻接顶点均已被访问，则退回到顶点D；

顶点D的邻接顶点均已被访问，退回到顶点C；顶点C的邻接顶点均已被访问，则退回到顶点B；顶点B的邻接顶点均已被访问，则退回到顶点A，顶点A为**起始顶点**，深度优先搜索结束。

上面所说的这些过程不正就是**递归**的过程吗？

为了更清楚的理解图的深度优先搜索和二叉树的前序遍历、中序遍历、后序遍历均属于一类方法，我们对最终的遍历结果图做一定的位置调整：



细心的一定发现，这就是我们的前序遍历过程呀！相信看到这里的一定对深度优先搜索豁然开朗了。

为了更加清楚图的深度优先搜索，我们将上面的过程总结为以下三个步骤：

- 1、首先选定一个未被访问过的顶点V作为起始顶点（或者访问指定的起始顶点V），并将其标记为已访问过；

- 2、然后搜索与顶点V邻接的所有顶点，判断这些顶点是否被访问过，如果有未被访问过的顶点W；再选取与顶点W邻接的未被访问过的一个顶点并进行访问，依次重复进行。当一个顶点的所有的邻接顶点都被访问过时，则依次回退到最近被访问的顶点。若该顶点还有其他邻接顶点未被访问，则从这些未被访问的顶点中取出一个并重复上述过程，直到与起始顶点V相邻接的所有顶点都被访问过为止。

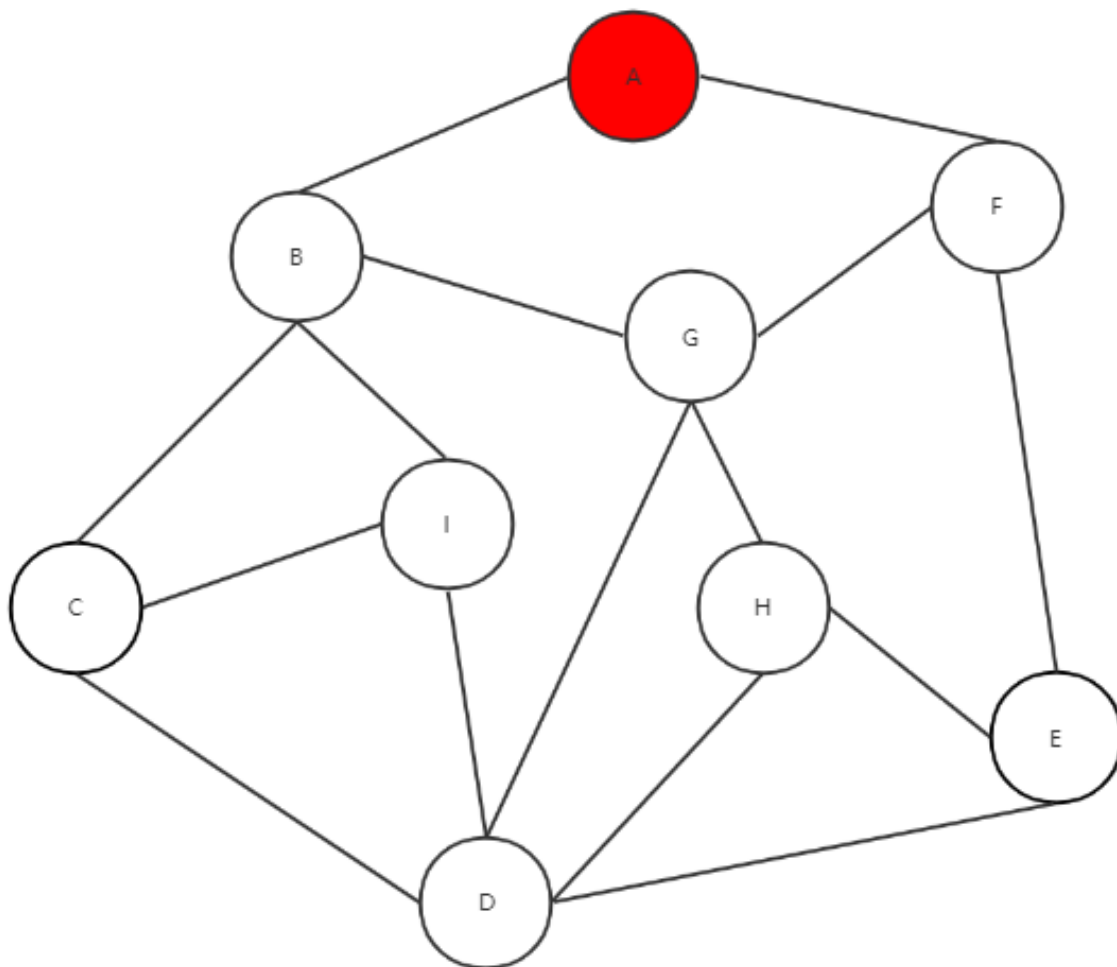
- 3、若此时图中依然有顶点未被访问，则再选取其中一个顶点作为起始顶点并进行遍历，转（2）。反之，则遍历结束。

广度优先搜索

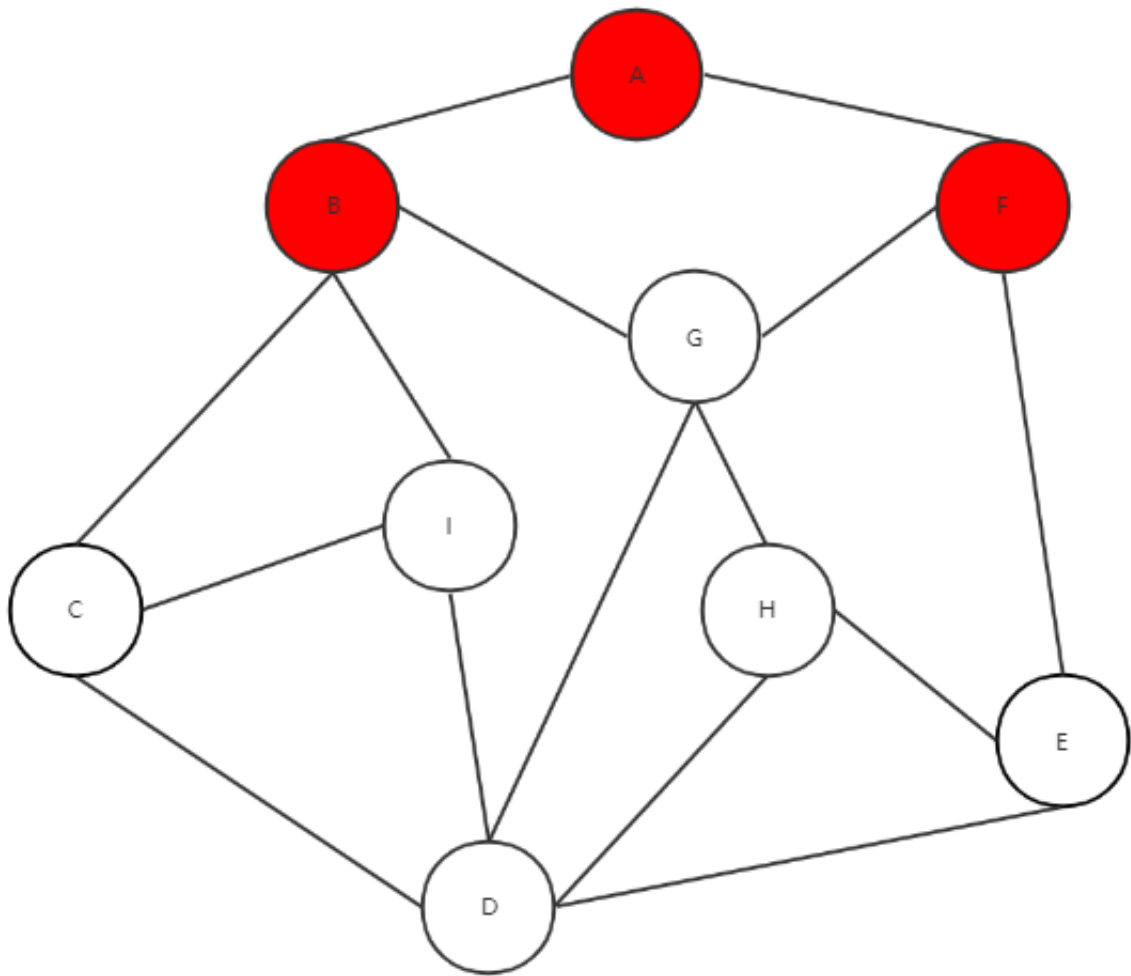
广度优先遍历（Breadth First Search），又称为广度优先搜索，简称**BFS**。树的层序遍历方式便是一种广度优先搜索方式。为了清晰地理解广度优先搜索，我们同样以深度优先搜索的例子一起走一遍，这不过，我们对图中顶点的位置做了调整，这样看起来更清楚。

假定从顶点A开始进行广度优先搜索，则遍历的顺序为：

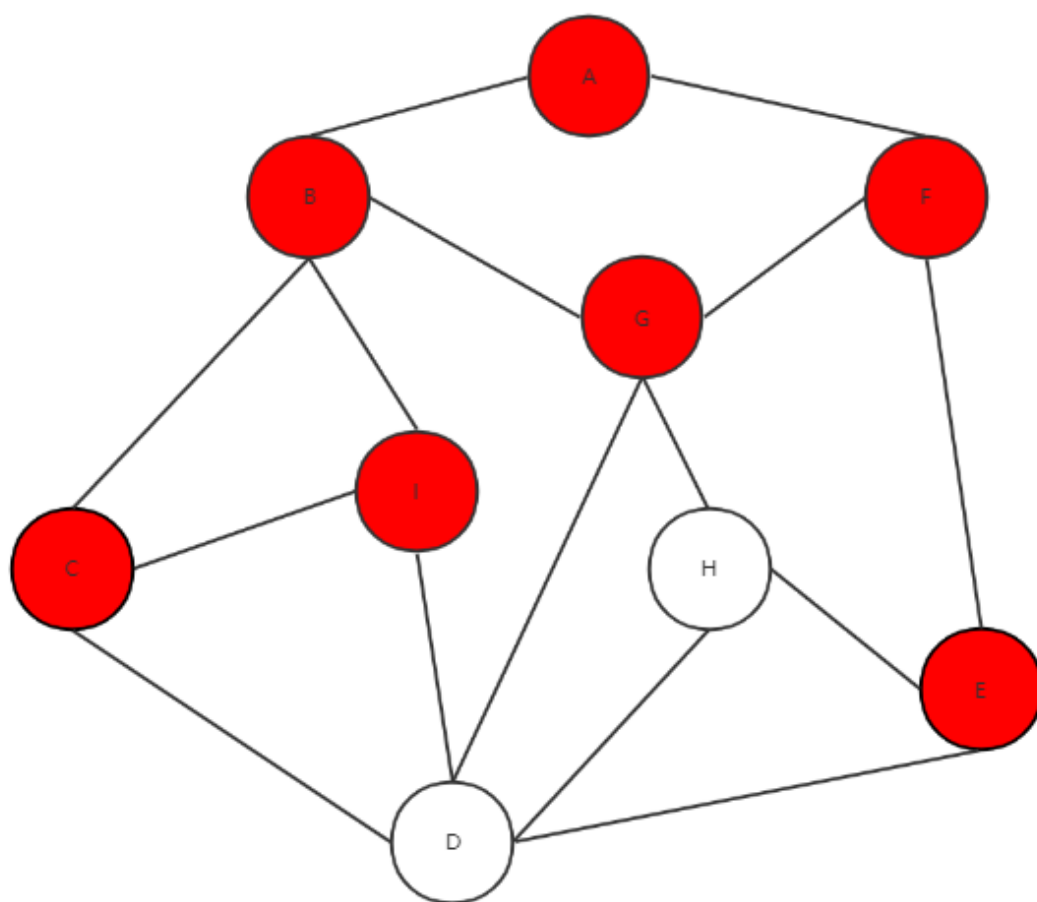
第一步：访问顶点A



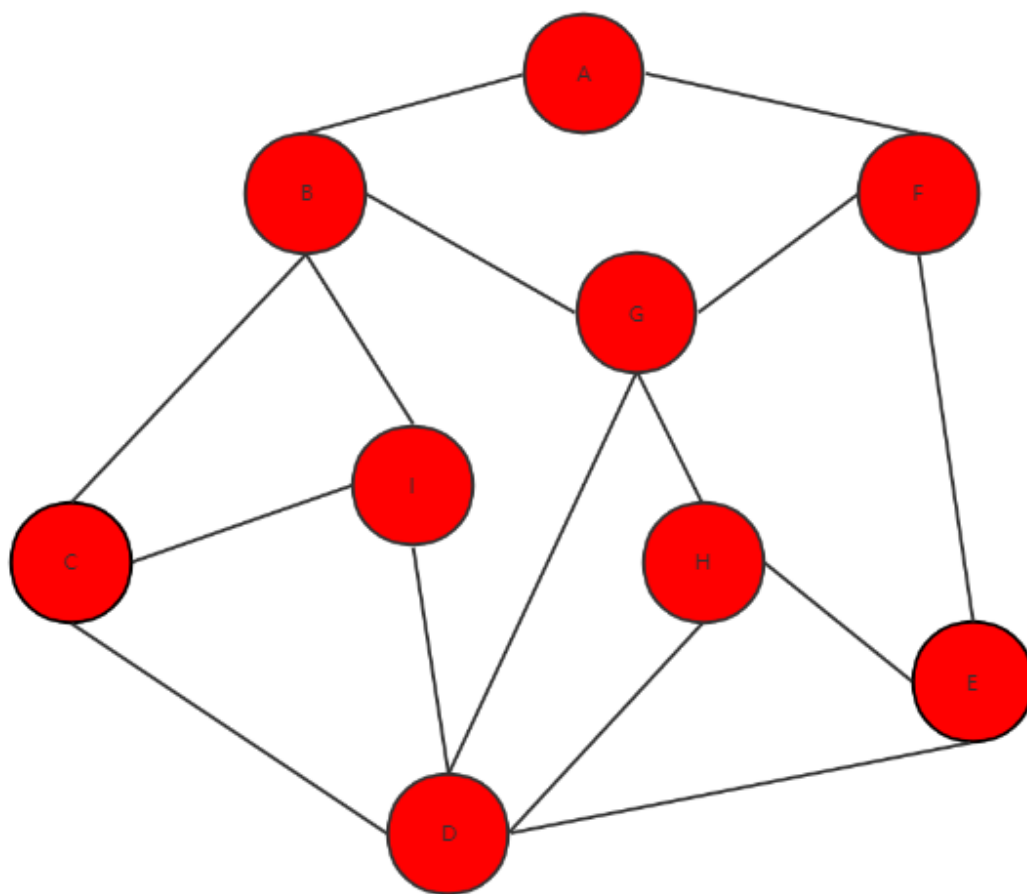
第二步：访问顶点A的所有未被访问的邻接顶点，顶点B和顶点F



第三步：访问顶点B和顶点F的所有未被访问的邻接顶点，顶点C、I、G、E；



第四步：访问顶点C、I、G、E 的所有邻接顶点中未被访问的顶点，顶点D和顶点H。



这个时候我们发现，广度优先遍历的步骤好少呀！

这里只是给你们展示了一层一层遍历的过程，并没有展示每一层具体如何被访问，这就要考虑到 **BFS** 的实现了。

那么要实现对图的广度优先遍历，显然和树的层序遍历一样，采用队列来实现。

总结

图的遍历方式包括深度优先搜索（DFS）和广度优先搜索（BFS），其中 DFS 使用递归或栈进行实现，而 BFS 则采用队列进行实现。对比树的四种遍历方式，前序遍历、中序遍历和后序遍历均类似于 DFS，而层序遍历类似于 BFS，前中后序也均可采用栈的方式进行实现，层序遍历可以采用队列的方式进行实现。

这样看来，知识的融会贯通多么重要，总体而言，掌握下面的两条链，你便可以解决好多问题。

DFS → 前中后序 → 栈 → 线性表

BFS → 层序遍历 → 队列 → 链表

BFS的缺点适用情况

BFS的一般步骤：

- 1、将初始点（一个或多个）加入队列
- 2、从队列头取出点，判断初始点的周边点，将符合条件的点加入队列尾部
- 3、重复2操作，直至队列为空。（一般每个点只加入队列一次）

BFS的缺点：

在搜索过程中，BFS必须要保存搜索过程中的状态，用于判重。

BFS的适用情况

一般在树或者图中，用BFS的可能性比较大。

BFS是用来搜索最短径路的解是比较合适的，比如求最少步数的解，最少交换次数的解，因为BFS搜索过程中遇到的解一定是离根最近的，所以遇到一个解，一定就是最优解，此时搜索算法可以终止。

这个时候不适宜使用DFS，因为DFS搜索到的解不一定是离根最近的，只有全局搜索完毕，才能从所有解中找出离根的最近的解。（当然这个DFS的不足，可以使用迭代加深搜索ID-DFS去弥补）

DFS的缺点以及适用情况

DFS的优点

空间优劣上，DFS是有优势的，**DFS不需要保存搜索过程中的状态**，而BFS在搜索过程中需要保存搜索过的状态，而且一般情况需要一个队列来记录。

因为根据栈的思想，DFS在搜索一个点以后，会弹出该点，就不需要保存已经搜索过的点。而BFS是必定保存搜索过的点的。

DFS的缺点

因为DFS含有栈的思想，因此经常用递归解决问题，但是如果不给递归限制深度，往往会超过时间与空间复杂度的。

二维数组的题目，N小于20的，适用DFS。而一般 $N \leq 200$ ， $N \leq 1000$ 这种，一定不可能用DFS去做。而且并不只是整个题目不能用DFS，其中的每一步也不能使用DFS。

上面的N指的应该是递归深度。

比如LeetCode中的Unique Paths，若是用大集合，会超时，该题适合用DP解。

DFS的适用情况

3.DFS适合搜索全部的解，因为要搜索全部的解，那么BFS搜索过程中，遇到离根最近的解，并没有什么用，所以搜索全部解的问题，DFS显然更加合适。

当求解目标，必须要走到最深（例如树，走到叶子节点）才能得到一个解，这种情况适合用深搜。

